

Modern Offensive Security — Exercises

Cloud, Identity, Kubernetes, CI/CD, Supply Chain & LLM Agents

Cyber Security Master Lecture

June 7, 2026

Authorization & ethics. Every hands-on task here uses an *intentionally vulnerable, sanctioned* range or *your own* account / repository / cluster. Do **not** enumerate, access, or test cloud storage, identity providers, clusters, pipelines, packages, or AI systems you do not own or are not explicitly permitted to test. Unauthorized cloud access is a criminal offence (§202a–c StGB / CFAA) and a terms-of-service violation. Mixed written/lab format: questions marked *[lab]* need the listed sandbox.

Sandboxes used (free, made to be attacked)

- **flaws.cloud** & **flaws2.cloud** — guided AWS S3/IAM challenges (browser + AWS CLI).
- **CloudGoat** (Rhino Security Labs) — deploy vulnerable AWS scenarios *into your own* account.
- **Kubernetes Goat** — intentionally vulnerable cluster (run locally in kind/minikube).
- **OWASP Juice Shop** — includes an OAuth flow to inspect.
- **Gandalf** (Lakera) / **Damn Vulnerable LLM Agent** — prompt-injection practice.
- A **GitHub repo you own** with GitHub Actions for the CI/CD tasks.

1. Cloud Asset Discovery & Exposed Storage

Exercise 1.1 (*surface*)

Explain why cloud adoption tends to *enlarge* an organisation's external attack surface. List three passive data sources you would use to enumerate a company's cloud-hosted assets without sending traffic to them, and what each yields. [6 pts]

Exercise 1.2 (*naming*)

Given the seed keyword **acme**, write five plausible storage bucket names an attacker would guess, and name the technique. Why is predictable naming a (defensive) liability? [4 pts]

Exercise 1.3 (*s3-flaws*)

[lab] Work the first two levels of **flaws.cloud**. For Level 1, give the exact command you used to list the public bucket without credentials, and explain why it succeeded. State the single AWS setting that would have prevented it. [8 pts]

Exercise 1.4 (*subdomain*)

Define a *subdomain takeover*. Describe the misconfiguration that enables it and one concrete impact. [4 pts]

2. OAuth 2.0 & OpenID Connect

Exercise 2.1 (*flows*)

Draw or describe the **Authorization Code flow with PKCE**. Label the role of **code**, **state**, the **code_verifier** / **code_challenge**, and the token exchange. Why is the implicit flow discouraged today? [8 pts]

Exercise 2.2 (*redirect*)

An app validates `redirect_uri` with a “starts-with” check against `https://app.example.com`. Show how an attacker abuses this (give a malicious `redirect_uri`) and what they obtain. State the correct validation rule. [6 pts]

Exercise 2.3 (*jwt*)

You receive this (decoded) ID token header: `{"alg":"none","typ":"JWT"}`. What attack does this enable, and what must the relying party check on *every* token (list four claims/checks)? [6 pts]

Exercise 2.4 (*scope*)

A single-page app requests scope `read write admin offline_access` for a feature that only reads a profile. Name the principle violated and two concrete risks of the over-broad grant. [4 pts]

3. Kubernetes Misconfigurations

Exercise 3.1 (*cani*)

[lab] In Kubernetes Goat (or any test cluster), run `kubectl auth can-i -list` for a low-privileged service-account token. Paste two permissions that stand out and explain the risk if an attacker controls a pod using that token. [6 pts]

Exercise 3.2 (*privesc*)

Explain how a **privileged** or `hostPath` pod allows escape to the underlying node. Give one Pod Security control that blocks it. [6 pts]

Exercise 3.3 (*imds*)

A pod can reach `http://169.254.169.254/`. What is this endpoint, what can an attacker retrieve, and how does that enable cloud-account compromise? Give one mitigation. [6 pts]

Exercise 3.4 (*secrets*)

A teammate says “our K8s Secrets are safe, they’re base64 encoded”. Correct them in two sentences and name the proper protection. [4 pts]

4. CI/CD Secrets & Supply Chain

Exercise 4.1 (*leak*)

[lab] In a repo you own, commit a fake secret (AKIA...), then remove it in a later commit. Run a secret scanner (e.g. `trufflehog`) over the history. Does deleting the file remove the secret? What is the correct remediation? [6 pts]

Exercise 4.2 (*ppe*)

Define **Poisoned Pipeline Execution**. Describe a scenario where a pull request from a fork leads to secret theft, and two controls that prevent it. [6 pts]

Exercise 4.3 (*oidc*)

Explain how **OIDC federation** between a CI provider and a cloud account removes the need for long-lived access keys, and why that is more secure. [4 pts]

Exercise 4.4 (*depconf*)

Explain **dependency confusion**. Why does publishing a public package with the same name as an internal one sometimes win? Give the two main defences. [6 pts]

Exercise 4.5 (*provenance*)

[lab] Generate an **SBOM** for a small project (`syft dir:.`) and scan it (`grype`). Report the count of findings and explain what an SBOM is good for beyond vuln scanning. [6 pts]

5. LLM & AI-Agent Security

Exercise 5.1 (*injection*)

[lab] Beat at least three levels of **Gandalf**. For one level, paste the prompt that worked and explain *why* it worked in terms of the model treating your input as instructions. [6 pts]

Exercise 5.2 (*indirect*)

Explain **indirect prompt injection** and why it is more dangerous for an *agent* (with tools) than for a chat-only LLM. Give a concrete confused-deputy scenario. [6 pts]

Exercise 5.3 (*output*)

An agent's reply is inserted unescaped into a SQL query / an HTML page / a shell command. Map each to the classic vulnerability class it re-introduces, and state the OWASP-LLM category. [6 pts]

Exercise 5.4 (*agency*)

List three controls that reduce **excessive agency** in a tool-using agent, and explain the “treat the LLM as an untrusted user” mental model. [6 pts]

Challenge (optional, bonus)

Exercise 5.5 (*threatmodel*)

Pick one system you actually use (a cloud app, a K8s service, or an LLM agent). Produce a one-page threat model: assets, trust boundaries, three plausible attack paths from this lecture, and the prioritized defences. Map each attack path to its MITRE ATT&CK (Cloud) or OWASP-LLM identifier. [10 bonus pts]